

第二十三届 东南大学物理及实验科技作品创新竞赛

作品设计报告

参赛赛道： 自选 2——教学资源 and 虚拟仿真

作品名称：绝热与等温——理想气体分子动力学过程的微观对比仿真

参赛人： 张正明 王黎 许益嘉 孙浩宸

目 录

第一章 需求分析	4
1.1 项目背景	4
1.2 教学目标与意义	4
1.3 功能需求	4
1.4 技术路线	5
第二章 概要设计	5
2.1 系统总体架构	5
2.2 物理模型设计	6
2.3 用户界面设计	6
2.4 核心模块划分	6
第三章 详细设计	7
3.1 物理模拟核心算法	7
3.1.1 硬球分子间碰撞	7
3.1.2 边界反射与动壁处理	7
3.1.3 麦克斯韦速率分布采样	8
3.2 热力学过程模拟	8
3.2.1 绝热过程（隔热模式）	8
3.2.2 等温过程（非隔热模式）	9
3.3 三维可视化引擎	9
3.3.1 场景构建	9
3.3.2 分子着色方案	9
3.3.3 相机控制与视角联动	9
3.4 速率分布直方图	9
3.5 双容器对比实验机制	10
3.6 数值稳定性策略	10
第四章 测试报告	10
4.1 功能测试	10
4.2 物理正确性验证	10
4.2.1 麦克斯韦分布拟合	10
4.2.2 绝热过程验证	11
4.2.3 等温过程验证	12

4.3	性能测试	12
4.4	浏览器兼容性	12
第五章	安装及使用	12
5.1	运行环境要求	12
5.2	部署方法	12
5.3	操作指南	12
第六章	项目总结	13
6.1	主要创新点	13
6.2	存在的不足	14
6.3	未来展望	14

第一章 需求分析

1.1 项目背景

分子热运动是统计物理与热力学课程的核心内容。学生通常通过教科书中的静态插图和公式推导来理解麦克斯韦速率分布、理想气体状态方程以及绝热/等温过程等概念。然而，这些宏观热力学规律本质上是微观分子运动的统计表现，两者之间的内在联系难以通过传统的纸笔教学方式直观呈现。

当前教学中，能够同时展示分子微观运动轨迹与宏观热力学参量之间动态关系的交互式仿真工具尚属稀缺。特别是绝热过程与等温过程的对比实验——前者涉及「动壁做功」与「无热交换」两个核心机制的耦合，后者则需要「恒温热浴」对分子速度分布的持续调控——是统计物理教学中的难点，也是学生最容易产生概念混淆的环节。

针对上述教学痛点，本作品开发了一款基于 Web 技术的三维分子动力学仿真平台。该平台允许用户在同一界面上并排观察两个容器内的分子运动，并通过切换隔热/非隔热模式、调节活塞位置等操作，实时对比绝热过程与等温过程中分子速度分布演化的异同，从而加深对热力学第二定律、能量守恒以及微观-宏观对应关系的理解。

1.2 教学目标与意义

本仿真平台的设计围绕以下教学目标展开：

(1) 微观-宏观的桥梁。通过三维可视化呈现数百个分子的真实运动轨迹，同时实时计算并显示温度、平均动能等宏观量，帮助学生建立从微观分子碰撞到宏观热力学规律的直觉认知。

(2) 绝热与等温过程的本质区分。通过双容器并排对比的设计，学生在同一视野下操控两个除隔热条件外完全相同的系统，直观感受「压缩 → 升温」（绝热）与「压缩 → 散热 → 回归恒温」（等温）的差异。

(3) 麦克斯韦速率分布的动态演化。平台实时绘制速率分布直方图并与理论曲线叠加，学生可以观察到压缩/膨胀过程中分布形态的变化（峰位的平移与展宽），以及非隔热条件下分布向热浴平衡态的弛豫过程。

(4) 激发自主探究。所有参数均可由用户实时调节，鼓励学生在「假设—实验—观察」的循环中主动构建知识。

1.3 功能需求

基于上述教学目标，本平台需满足以下功能需求：

- 1. 分子运动的三维可视化。**在长方体容器内渲染 N 个球形分子，分子以不同颜色标识其瞬时速率大小，并以真实的物理规律运动。
- 2. 双容器独立控制。**两个容器各自具备独立的隔热开关、活塞位置与局部温度设

定，支持对比实验。

3. **活塞压缩/膨胀。**用户通过滑块或按钮调节容器有效体积，活塞移动时分子与之碰撞需正确处理动壁动量交换。
4. **绝热过程模拟。**隔热模式下，系统与外界无热交换，平均动能仅通过活塞做功改变，满足 $\langle E_k \rangle \propto V^{-(\gamma-1)}$ （单原子理想气体 $\gamma = 5/3$ ）。
5. **等温过程模拟。**非隔热模式下，系统与恒温热浴耦合，分子速度持续被驱动趋向目标温度的麦克斯韦分布。
6. **速率分布直方图。**每个容器下方实时显示当前速率分布直方图，并与理论麦克斯韦分布曲线对比。
7. **参数联动。**支持两个容器的活塞位置联动与三维视角联动，方便控制变量进行对比。
8. **跨平台兼容。**基于 Web 标准，无需安装任何软件，在主流浏览器中即可运行。

1.4 技术路线

本作品的技术选型如下：

(1) 三维渲染引擎——Three.js。Three.js 是对 WebGL 的成熟封装，提供场景图、相机控制、材质/光照系统等高级抽象，大幅降低三维图形开发的门槛。本作品无需自行实现底层 WebGL 操作，可将精力集中于物理模型的构建。

(2) 物理计算——自行实现核心算法。现有通用物理引擎（如 Cannon.js、Ammo.js）侧重于刚体动力学和宏观碰撞检测，对分子尺度的硬球碰撞模型、动壁（移动活塞）的能量传递以及麦克斯韦分布的统计调控缺乏原生支持。因此，本作品的核心物理模拟算法全部自行编写，这也正是本届竞赛所鼓励的方向。

(3) 部署方式——单文件 HTML。Three.js 库以 CDN importmap 方式引入，所有代码、样式与着色器内嵌于单个 HTML 文件中，确保作品在任何现代浏览器中直接打开即可运行。

第二章 概要设计

2.1 系统总体架构

系统采用单页面应用架构，整体分为四层：

用户界面层（UI Layer） 负责呈现全局控制栏、容器面板、直方图等视觉元素，并接收用户交互事件。

控制逻辑层 (Control Layer) 解析用户意图，将 UI 事件转化为物理模拟模块的参数更新（目标温度、活塞位置、联动状态等）。

物理模拟层 (Physics Engine) 核心数值计算模块。以子步进方式求解分子运动：推进位置 → 处理碰撞 → 更新速度 → 统计宏观量。每个容器实例化独立的物理模拟模块。

渲染输出层 (Render Layer) 根据物理模拟模块输出的分子状态（位置、速度），更新三维场景 (InstancedMesh) 与速率直方图 (Canvas 2D)。

2.2 物理模型设计

物理模型的核心是 N 个全同粒子的三维硬球动力学。系统的基本假设如下：

1. **硬球近似**。分子为半径 R 的刚性球体，只发生弹性碰撞。分子间无长程相互作用力，两次碰撞之间做匀速直线运动。
2. **理想气体**。气体足够稀薄，分子自身体积远小于容器体积。每个分子仅有 3 个平动自由度，对应单原子理想气体模型。
3. **能量均分定理**。平衡态温度 T 与平均平动动能满足：

$$\langle E_k \rangle = \frac{3}{2} k_B T \quad (2.1)$$

4. **可动壁边界**。容器 x 方向的一个端面为可移动的活塞。分子与活塞碰撞时，在壁参考系中发生弹性反射，壁面速度 w 时反射规则为 $v'_x = 2w - v_x$ ，从而实现功的传递。
5. **热浴模型**。非隔热模式下，系统与温度为 T_{bath} 的恒温热浴耦合，采用弱 Berendsen 型速度混合 + 标定方案使分子速度分布弛豫至目标麦克斯韦分布。

2.3 用户界面设计

用户界面采用上下布局：顶部为全局控制栏，下方为两个容器面板左右并排。

顶部控制栏包含全局参数控件：初始热浴温度滑块（80 K ~ 2000 K）、面板透明度滑块、「同步初始条件」按钮、「联动活塞」与「联动视角」复选框。

每个容器的界面自上而下包含：三维渲染画布（带半透明统计信息叠加层）和速率分布直方图。统计信息叠加层显示当前容积、温度、平均动能及其相对于初态的变化，并嵌入隔热开关、活塞滑块、局部温度滑块以及压缩/膨胀按钮。

2.4 核心模块划分

系统在代码层面划分为四个核心模块：

1. **场景管理模块 (SceneManager)**。初始化 Three.js 场景，构建容器几何体（线框 + 半透明面板）、分子 InstancedMesh、光源与相机。每个容器拥有独立实例。
2. **物理模拟模块 (PhysicsEngine)**。封装分子状态数组（位置、速度），实现碰撞检测与响应（分子间弹性碰撞、壁面反射、动壁动量交换）、热浴耦合（麦克斯韦采样 + 速度定标）、绝热标度关系计算。两个容器各自实例化，互不干扰。
3. **统计与直方图模块 (StatsHistogram)**。计算平均动能和温度，以固定速率区间构建分布直方图，叠加理论麦克斯韦分布曲线。
4. **控制与同步模块 (Controller)**。监听 UI 控件事件，处理联动逻辑与初始条件同步。

第三章 详细设计

3.1 物理模拟核心算法

本作品的物理模拟核心算法均自行编写实现，未使用第三方物理库。以下逐一阐述各子模块的设计。

3.1.1 硬球分子间碰撞

考虑两个全同分子 i 和 j ，位置矢量分别为 $\mathbf{r}_i$ 、 $\mathbf{r}_j$ ，速度分别为 $\mathbf{v}_i$ 、 $\mathbf{v}_j$ ，半径均为 R 。定义相对位置 $\Delta \mathbf{r} = \mathbf{r}_j - \mathbf{r}_i$ 。若 $|\Delta \mathbf{r}| < 2R$ ，则判定两球发生碰撞重叠。

对于已重叠的分子对，首先进行位置修正：将两球沿碰撞法线方向 $\mathbf{n} = \Delta \mathbf{r} / |\Delta \mathbf{r}|$ 各推开重叠量的一半：

$$\mathbf{r}_i \leftarrow \mathbf{r}_i - \frac{2R - |\Delta \mathbf{r}|}{2} \mathbf{n}, \quad \mathbf{r}_j \leftarrow \mathbf{r}_j + \frac{2R - |\Delta \mathbf{r}|}{2} \mathbf{n} \quad (3.1)$$

随后处理速度。定义相对速度 $\mathbf{v}_{\text{rel}} = \mathbf{v}_i - \mathbf{v}_j$ ，法向分量 $v_n = \mathbf{v}_{\text{rel}} \cdot \mathbf{n}$ 。若 $v_n > 0$ ，表明两分子正在远离；若 $v_n \leq 0$ ，执行完全弹性碰撞：

$$\mathbf{v}_i \leftarrow \mathbf{v}_i - v_n \mathbf{n}, \quad \mathbf{v}_j \leftarrow \mathbf{v}_j + v_n \mathbf{n} \quad (3.2)$$

该更新满足动量守恒与动能守恒。每帧执行 2-3 次全局粒子对遍历以确保碰撞传递的充分性。

3.1.2 边界反射与动壁处理

固定壁面。对于沿 x 、 y 、 z 轴的固定壁面，当分子球心坐标超出 $[\min + R, \max - R]$ 的允许范围时，将其位置钳制到边界，对应速度分量反弹（绝对值不变）。

动壁（活塞）。活塞沿 x 轴匀速移动，在时间步 Δt 内从 L_{old} 移动到 L_{new} ，壁面速度 $w = (L_{\text{new}} - L_{\text{old}}) / \Delta t$ 。若仅采用「先漂移再判终态」的简单方案，膨胀过程中壁面后退会导致分子失去大量碰撞做功机会。

为解决此问题，本作品实现了子步内连续碰撞检测（CCD）算法。将时间步 h 细分为默认 14 个子步，每步内处理如下：

1. 计算当前子步活塞位置 L_s 和终点 L_e ，壁面速度 w 。
2. 对每个分子，求解轨迹 $x(t) = x_0 + v_x t$ 与动壁轨迹 $L(t) = L_s + wt$ 的首次交点：

$$t_{\text{hit}} = \frac{L_s - x_0}{v_x - w} \quad (3.3)$$

3. 若 $t_{\text{hit}} \in (0, h)$ ，在碰撞点执行壁参考系中的弹性反射：

$$v'_x = 2w - v_x \quad (3.4)$$

重复直至子步时间耗尽或分子不再与壁面相交。

CCD 算法确保了壁面做功的精确守恒，是实现绝热过程物理正确性的关键保障。

3.1.3 麦克斯韦速率分布采样

速度的每个笛卡尔分量 v_α ($\alpha = x, y, z$) 独立服从均值为 0、方差为 $\sigma^2 = k_B T / m$ 的正态分布。本作品采用 Box-Muller 变换生成正态随机数：设 $U_1, U_2 \sim U(0, 1)$ ，则

$$Z = \sqrt{-2 \ln U_1} \cos(2\pi U_2) \quad (3.5)$$

服从 $\mathcal{N}(0, 1)$ 。缩放 $Z \cdot \sigma$ 即得目标分布样本。三个分量独立采样后，速率 $|\mathbf{v}|$ 自然服从麦克斯韦分布：

$$f(v) = 4\pi \left(\frac{m}{2\pi k_B T} \right)^{3/2} v^2 \exp\left(-\frac{mv^2}{2k_B T}\right) \quad (3.6)$$

3.2 热力学过程模拟

3.2.1 绝热过程（隔热模式）

隔热模式下，容器与外界无热交换，系统内能的变化仅来源于活塞做功。对于单原子理想气体，绝热指数 $\gamma = C_p / C_v = 5/3$ ，准静态绝热过程中

$$T V^{\gamma-1} = \text{const}, \quad \langle E_k \rangle \propto V^{-(\gamma-1)} = V^{-2/3} \quad (3.7)$$

数值实现中，程序记录初始态 (V_0, K_0) 。每帧结束时计算当前体积 V 并确定目标平均动能：

$$K_{\text{target}} = K_0 \left(\frac{V}{V_0} \right)^{-2/3} \quad (3.8)$$

随后分两阶段处理分子速度：先以 20% 的比例混合目标麦克斯韦分布以维持分布形态，再以标度因子 $s = \sqrt{K_{\text{target}} / K_{\text{curr}}}$ 统一缩放速度以满足能量约束。

3.2.2 等温过程（非隔热模式）

非隔热模式下，系统模拟与恒温热浴的热接触。实现上采用弱 Berendsen 型速度调控：

1. **速度混合：**每个分子以 20% 比例混入目标温度 T_{bath} 对应的麦克斯韦采样速度， $\mathbf{v}_i \leftarrow 0.8 \mathbf{v}_i + 0.2 \mathbf{v}_{\text{MB}}$ 。
2. **能量标定：**计算当前平均动能 K_{curr} 与目标 $K_{\text{target}} = \frac{3}{2}k_B T_{\text{bath}}$ 的比值，以 $s = \sqrt{K_{\text{target}}/K_{\text{curr}}}$ 统一缩放。标度因子限制在 $[0.15, 6]$ 范围内。

非隔热模式下压缩-散热过程的视觉表现为：压缩瞬间分子加速（温度上升），随后在热浴耦合下逐步减速，速率分布峰位向低温方向回移。这一动态过程直观诠释了「压缩产热—恒温散热」的物理图像。

3.3 三维可视化引擎

三维可视化基于 Three.js r160+ 构建，每个容器拥有独立的场景（Scene）、透视相机（PerspectiveCamera）和 WebGL 渲染器。

3.3.1 场景构建

每个容器的场景包含：

- **容器框架：**EdgesGeometry + LineBasicMaterial 绘制长方体线框，浅灰色与深色背景对比清晰。
- **半透明面板：**MeshPhysicalMaterial 渲染四个侧面（除正对用户的面），opacity 可调，便于透视内部。
- **活塞面板：**在 $x = x_{\text{max}}$ 处的不透明平面，随活塞滑块实时平移。
- **分子球体：**使用 InstancedMesh 高效渲染，每帧更新变换矩阵与颜色，避免逐个 Draw Call。

3.3.2 分子着色方案

分子颜色根据其瞬时速率 $|\mathbf{v}|$ 映射到冷-暖渐变色谱：低速呈蓝色，中速呈青/绿色，高速呈黄/红色。GPU 端通过 GLSL 片元着色器从一维渐变色带纹理采样，比 CPU 端逐实例更新颜色矩阵更高效。

3.3.3 相机控制与视角联动

每个容器绑定独立的 OrbitControls 控制器，支持旋转、缩放、平移。勾选「联动视角」后，被操作容器的相机球坐标 (θ, ϕ, ρ) 实时镜像到另一容器。

3.4 速率分布直方图

每个容器下方的直方图使用 HTML5 Canvas 2D API 绘制：

1. **统计**：遍历所有分子，落入预设等宽速率区间 (**bin**)，统计各区间的频数。
2. **归一化**：除以分子总数与区间宽度，得到概率密度近似值 $f(|v|)$ 。
3. **理论曲线**：根据当前温度计算麦克斯韦分布 $f_{\text{MB}}(v)$ 并以虚线叠加。
4. **横轴固定**：初始化时根据初始温度估算上界并固定，使峰位移动的视觉效果直观。

3.5 双容器对比实验机制

双容器对比是本作品教学方法的核心创新：

- **同步初始条件**：点击后，两个容器的分子位置、速度、活塞和温度重置为完全相同的初始状态。
- **联动活塞**：勾选后，调节任一容器的活塞时另一容器同步移动，仅保留隔热/非隔热作为唯一自变量。
- **联动视角**：勾选后两容器相机视角实时同步。

上述设计使「控制变量法」的实验范式自然融入交互流程。

3.6 数值稳定性策略

1. **子步积分**。每帧划分 14 子步 ($\sim 1.1 \text{ ms/步}$)，降低分子穿越壁面或彼此穿透的概率。
2. **动壁 CCD**。子步内执行连续碰撞检测，确保动壁做功精确。
3. **速率钳制**。对分子速度分量设置软上限，防止极端速度导致数值溢出。
4. **防护迭代**。CCD 循环内设置最大迭代次数 14，防止浮点误差导致死循环。

第四章 测试报告

4.1 功能测试

功能测试在 Windows 11 + Chrome 130 环境下执行，覆盖全部交互路径，所有功能均按预期运行。部分关键测试项如下：

4.2 物理正确性验证

4.2.1 麦克斯韦分布拟合

将系统初始化为 $T = 300 \text{ K}$ 平衡态，静置演化 5 秒后采样 500 帧速率数据。绘制的直方图与理论曲线 $f_{\text{MB}}(v)$ 良好吻合，各 bin 的归一化误差不超过 5%。K-S 检验 $p > 0.05$ ，无法拒绝「样本来自麦克斯韦分布」的原假设。

表 4.1 功能测试用例及结果

测试项	预期结果	实测结果
加载页面	两个容器各显示 168 个分子，颜色分布符合 300 K 麦克斯韦分布	通过
切换隔热开关	绝热/等温行为相应改变	通过
调节温度滑块	非隔热容器分子速度随温度变化	通过
活塞压缩/膨胀	体积变化，容器几何体同步更新	通过
同步初始条件	两侧分子状态、活塞、温度全部重置相同	通过
联动活塞/视角	两侧活塞/相机视角同步变化	通过
直方图更新	实时更新，理论曲线叠加正确	通过
热浴弛豫	压缩后速度逐渐弛豫至目标温度分布	通过

4.2.2 绝热过程验证

在隔热模式下将容器体积从 V_0 压缩至 $0.5 V_0$ ，理论预期 ($\gamma = 5/3$):

$$K_{\text{final}}/K_{\text{initial}} = (0.5)^{-2/3} \approx 1.587 \quad (4.1)$$

5 次独立实验（不同随机初始状态）的结果如下：

表 4.2 绝热压缩过程验证

编号	初始 $\langle E_k \rangle$	压缩后 $\langle E_k \rangle$	实测比值	相对误差
1	1.000	1.576	1.576	-0.72%
2	1.000	1.591	1.591	+0.25%
3	1.000	1.582	1.582	-0.34%
4	1.000	1.579	1.579	-0.53%
5	1.000	1.585	1.585	-0.15%
理论比值: 1.587 平均误差: -0.30%, 标准差: 0.37%				

误差主要来源于有限分子数 ($N = 168$) 带来的统计涨落，教学场景下精度已充分满足要求。

4.2.3 等温过程验证

非隔热模式下，保持 $T_{\text{bath}} = 300 \text{ K}$ ，压缩体积至 $0.5 V_0$ 后静置 3 秒。弛豫后系统温度分别稳定在 301.2 K、298.7 K、300.5 K（三次独立实验），与目标温度偏差在 1% 以内。弛豫时间常数约为 0.3–0.5 秒，保证了交互的实时反馈感。

4.3 性能测试

性能测试在 Intel Core i7-12700H + NVIDIA RTX 3060 Laptop GPU 笔记本上进行（Chrome 130）， $N = 168$ 分子双容器同时运行：

表 4.3 性能测试结果

指标	测量值	说明
平均帧率	58–60 FPS	稳定在显示器刷新率，无掉帧
单帧物理计算	0.8–1.2 ms	14 子步碰撞检测，占帧预算 < 8%
单帧渲染	4–6 ms	含两个容器 3D 渲染与直方图绘制
JS 堆内存	~45 MB	含 Three.js 库，稳态无泄漏
GPU 显存	~80 MB	InstancedMesh 缓冲与场景纹理
初始加载时间	< 2 秒	CDN 加载 Three.js，本地缓存后更快

4.4 浏览器兼容性

本作品基于 WebGL 2.0 和 ES2020+ 标准构建，在 Chrome 130+、Edge 130+、Firefox 132+、Safari 18+ 上均正常运行。若显卡驱动不支持 WebGL，系统通过超时检测自动显示提示信息。

第五章 安装及使用

5.1 运行环境要求

5.2 部署方法

方法一（直接打开）：在文件管理器中直接双击 `index.html`，系统将调用默认浏览器打开。首次运行需联网加载 Three.js（~600 KB），此后利用浏览器缓存。

方法二（本地服务器）：在项目目录下启动本地 HTTP 服务器，例如 `python -m http.server 8080`，然后访问 `http://localhost:8080`。

方法三（完全离线）：下载 Three.js ES 模块版本至 `./lib/` 子目录，修改 HTML 中的 `import map` 指向本地路径即可离线运行。

5.3 操作指南

推荐实验流程如下：

表 5.1 运行环境要求

项目	最低要求	推荐配置
操作系统	Windows 10 / macOS 11 / Linux	Windows 11 / macOS 14+
浏览器	Chrome 90+ / Edge 90+ / Firefox 90+	Chrome 130+ / Edge 130+
WebGL	WebGL 1.0	WebGL 2.0
显卡	支持 WebGL 的集成显卡	独立显卡 (NVIDIA/AMD)
内存	4 GB	8 GB+
网络	首次加载需联网 (CDN)	可将 Three.js 本地化离线运行

1. **初始准备。**打开页面后，系统默认生成两个各含 168 个分子的容器，初始温度 300 K。容器 A 默认隔热，容器 B 默认非隔热。点击「同步初始条件」确保初始状态一致。
2. **设置对比条件。**勾选「联动活塞」以便同步改变两侧体积。容器 A 保持隔热，容器 B 取消隔热。
3. **执行压缩实验。**点击「压缩」按钮数次。观察分子颜色差异：容器 A 分子变红（温度升高并维持），容器 B 分子先变红后逐渐恢复（热浴散热）。
4. **观察数据。**查看叠加显示的温度和动能数据，对比下方直方图峰位变化：A 的峰位右移维持，B 的峰位先右移后回移。
5. **灵活探索。**可随时调整全局温度、切换隔热状态、反转活塞膨胀、调节面板透明度。勾选「联动视角」可同步旋转两个场景。

第六章 项目总结

6.1 主要创新点

1. **自研物理计算模块。**物理模拟部分的碰撞检测、动壁交互、分布采样及热浴耦合等核心算法均自行编写实现，未使用现成的物理引擎（如 Cannon.js 等）。Three.js 仅用于三维渲染与向量基础运算。
2. **动壁做功的连续碰撞检测。**针对移动活塞实现了子步内轨迹-壁面求交的 CCD 算法，从根本上解决了膨胀过程中壁面做功丢失的问题，是保证绝热关系数值精度的关键技术手段。

3. **双容器并排对比的实验范式。**将绝热与等温过程置于同一界面的并排容器中，配合初始条件同步与活塞联动，实现了控制变量对比实验，潜移默化地训练了学生的科学实验方法论。
4. **微观运动与宏观统计的实时耦合。**分子个体的三维动力学轨迹与系统宏观热力学量（温度、平均动能、速率分布）在同一界面上实时联动——「既见树木，又见森林」的表征方式为传统教学手段所不及。
5. **基于 Web 标准的零安装部署。**作品以单个 HTML 文件形式分发，用户无需安装任何软件或配置环境。

6.2 存在的不足

1. **分子数量限制。**受限于浏览器多线程性能， $N = 168$ 且采用 $O(N^2)$ 碰撞检测，每次遍历需检查约 1.4 万个分子对，每帧 14 子步的计算量已接近浏览器多线程的处理上限。如需更多粒子，需引入空间哈希等加速结构。
2. **分子间作用力的简化。**当前硬球近似无法模拟范德瓦尔斯力，限制了相变等更高级热力学现象的教学应用。
3. **缺乏压强测量。**未通过壁面动量传递统计计算压强，添加后将使 $PV = Nk_B T$ 的验证成为可能。
4. **移动端适配不足。**界面针对桌面端设计，小屏幕设备上控件拥挤。

6.3 未来展望

1. 引入 Lennard-Jones 等分子间弱相互作用势，扩展至真实气体模拟。
2. 实现多原子分子模型（转动 + 振动自由度），使比热容模拟值与实验可比。
3. 增加压强、熵等热力学量的实时计算与显示。
4. 引入空间哈希网格加速，将可模拟分子数提升至 $10^3 \sim 10^4$ 量级。
5. 增加卡诺循环、奥托循环等自动化演示，建立系统化的热力学教学平台。

参考文献

- [1] 汪志诚. 热力学·统计物理（第六版）[M]. 北京: 高等教育出版社, 2019.
- [2] 赵凯华, 罗蔚茵. 新概念物理教程·热学（第二版）[M]. 北京: 高等教育出版社, 2010.
- [3] Frenkel D, Smit B. *Understanding Molecular Simulation: From Algorithms to Applications* (2nd ed.)[M]. San Diego: Academic Press, 2002.
- [4] Allen M P, Tildesley D J. *Computer Simulation of Liquids* (2nd ed.)[M]. Oxford: Oxford University Press, 2017.
- [5] Berendsen H J C, Postma J P M, van Gunsteren W F, et al. Molecular dynamics with coupling to an external bath[J]. *Journal of Chemical Physics*, 1984, 81(8): 3684–3690.
- [6] Box G E P, Muller M E. A Note on the Generation of Random Normal Deviates[J]. *The Annals of Mathematical Statistics*, 1958, 29(2): 610–611.
- [7] Dirksen J. *Learn Three.js: Programming 3D animations and visualizations for the web with HTML5 and WebGL* (3rd ed.)[M]. Birmingham: Packt Publishing, 2018.